A REFERENCE GUIDE FOR CLAUDE CODE · 2026

3 Claude Code Design Skills

Emil Kowalski's skill, Impeccable, and Taste Skill — what each one actually does, exact install commands, and how to stack all three without them fighting each other.

3	1	0
open-source skills, independently built	shared problem they all solve — AI slop	cost to install — all free and open-source

Sanchit Pandey

AI systems, automation & web development
Lucknow, India

START HERE

"AI slop" has a specific look

Inter font. A purple-to-blue gradient. Cards nested inside cards. A hero with a big number, a small label, and a glowing accent blob. Every model trained on the same corner of the internet produces the same handful of tells — and once you notice them, you cannot unsee them on other people's "AI-built" sites either.

This is not a model-quality problem. Left to its defaults, Claude is technically capable of good design and statistically biased toward generic design, because generic is what shows up most in training data. A design skill exists to counter that bias — it hands the model a specific, opinionated point of view instead of an average one.

WHAT THIS DOCUMENT IS

A verified profile of three real, independently-maintained, open-source skills, with exact install commands, what each is actually good at, and a stacking order that avoids them stepping on each other. Nothing here is affiliated with Anthropic or with the three creators — check each project's own repository for anything that changed since.

The three, in one line each

Skill	Creator	What it's for
Emil Kowalski's skill	Emil Kowalski — design engineer at Linear, creator of Sonner and Vaul	Motion and interaction: when to animate, when not to, and exactly how
Impeccable	Paul Bakaus — former Google Developer Advocate, creator of jQuery UI	Whole-interface polish: typography, color, spacing, layout, via named commands
Taste Skill	Leon Lin (LeonxInx)	Overall aesthetic direction: layout, hierarchy, and a named visual style before you build

They overlap in places and are strongest in different places. Section 06 covers how to combine them; read the individual profiles first.

BEFORE YOU INSTALL

A skill is software, not a setting

This matters more here than in most "here are 3 tools" lists, so it comes before the profiles rather than as a footnote.

A Claude Code skill is a folder of instructions Claude reads and follows — some skills also bundle scripts that get executed. Anthropic's own guidance on this is direct: treat installing a skill like installing software, because a poorly-vetted one could misuse tools or expose data, the same as any other code you did not write yourself.

- [ ] Open the repository before installing and skim the SKILL.md — it is plain markdown, readable in two minutes
- [ ] Prefer `npx skills add` or a git clone over piping an install script straight into bash (`curl ... | bash`) when an alternative exists
- [ ] Install into a project you can afford to review, not directly into something in production
- [ ] Check who maintains it and how recently — all three in this guide are active and reasonably well-known, which is itself a signal worth checking for any skill you add later
- [ ] Remember a skill can see and edit your codebase. Nothing here needs excessive permissions to do its job — be more careful with anything that asks for more

> None of this is a reason to avoid these three — all three are widely used, source-available, and reviewed in this document from their own repositories. It is a habit worth having for every skill you add after these, including ones you find on a random list with no context.

SKILL 01

Emil Kowalski's skill

> "The goal is not to animate for animation's sake, it's to build great user interfaces." — the philosophy the whole skill is built around.

Who built it

Emil Kowalski is a design engineer at Linear (previously Vercel), creator of the open-source Sonner (toasts) and Vaul (drawers) libraries, and author of the "Animations on the Web" course. The skill is a distillation of that course into rules an agent can follow.

What it actually does

This is not one skill but a small family, installed together. Each does a distinct job:

Skill name	Job
emil-design-eng	The main skill — animation plus some general design advice
review-animations	Strict review of existing animation code against a defined craft bar
improve-animations	Audits a whole codebase and returns a prioritised, executable fix plan
find-animation-opportunities	Finds places that would genuinely benefit from motion — and flags what should stay still
prototype	Builds several genuinely different versions of one UI piece to compare live
animation-vocabulary	A reverse-lookup glossary — turns "the bouncy thing" into its real term, so your prompts get more precise

The core rules it enforces

— Knowing when NOT to animate is treated as the primary skill, not an afterthought — a tool used constantly (like Raycast) benefits from near-zero animation; users with a clear goal do not want to be delighted, they want speed

— UI animations generally stay under 300ms. A 180ms transition reads as more responsive than a 400ms one — when in doubt, go faster

— Animate only transform and opacity — GPU-accelerated properties. Animating width, height, margin or top triggers expensive layout recalculation

— Interruptibility matters. Rapidly-triggered motion (toasts, toggles, drags) must retarget from its current state, not restart from zero

— prefers-reduced-motion is always respected, gently — reducing movement, not deleting all feedback

Install

TERMINAL

```
npx skills add emilkowalski/skill

# Installs all skills in the family. To use one in a session:
# just describe the task - e.g. "review the animations on this
# page" triggers review-animations automatically.
```

SKILL 02

Impeccable

> Built on top of Anthropic's own official frontend-design skill, then extended with deeper domain coverage and hard constraints against the patterns every model defaults to.

Who built it

Paul Bakaus — former Google Developer Advocate and creator of jQuery UI. Impeccable became one of the most widely adopted design skills in the Claude Code ecosystem within weeks of release.

What it actually does

Impeccable is the most comprehensive of the three — a full command vocabulary rather than a single always-on style. It ships with roughly 20+ commands, seven domain-specific reference files (typography, color, motion, spatial, interaction, responsive, UX writing), and a set of curated anti-patterns that tell the model exactly what to avoid.

The commands worth knowing

Command	What it does
/impeccable init	One-time setup — asks whether the surface is brand (marketing, landing) or product (app, dashboard), then writes PRODUCT.md and DESIGN.md so every later command has context
/polish	Refines an existing UI against all seven design pillars in one pass
/audit	Reviews the interface and reports what is off, without changing anything yet
/critique	A genuine UX read — visual hierarchy, information architecture, emotional read
/distill	Simplifies an interface that has accumulated visual noise
/animate	Applies motion following the same design pillars
/bolder / /quieter	Pushes the whole interface's intensity up or down in one command
/extract	Pulls reusable components and design tokens out of existing code into a system
/overdrive	A more aggressive redesign pass for when incremental polish is not enough

What it is actively avoiding

- × Inter as the default typeface for everything
- × Purple-to-blue gradients as the default accent treatment
- × Cards nested inside cards

- × Gray text on a colored background
- × The rounded-square icon tile sitting above every section heading

Install

TERMINAL

```
npx skills add pbakaus/impeccable

# Then, inside a Claude Code session in your project:
/impeccable init
# answers a few questions, writes PRODUCT.md and DESIGN.md

# From then on, in any session:
/polish
/audit
/critique
```

SKILL 03

Taste Skill

> "AI agents need taste, and taste can be codified." The broadest of the three — less a command set, more a transplanted point of view.

Who built it

Leon Lin (Leonxlnx). Free and open-source under MIT license, with a public research folder documenting why AI-generated interfaces tend toward "laziness" — cognitive shortcuts, output limits, and training-data bias — and how the skill's rules push against each cause specifically.

What it actually does

Where Impeccable gives you named commands to run at specific moments, Taste Skill is closer to a standing point of view the agent carries through a whole build — layout decisions, type scale, spacing rhythm, hierarchy and motion direction, applied continuously rather than as a discrete polish pass.

What makes it distinct from the other two

- Named aesthetic directions you can request explicitly — brutalist, minimalist, soft, editorial, premium-brand — rather than one house style
- Adjustable dials inside key skills, including `MOTION_INTENSITY` (hover-only through scroll/magnetic effects) and `VISUAL_DENSITY` (spacious through dense dashboard)
- Framework-agnostic by design — the rules target design intent, not a specific React or Tailwind API, so they hold up across stacks
- A companion image-generation skill that can produce reference frames first, then hand them to Claude Code, Cursor or Codex for implementation — useful when you want to see a direction before any code is written

Install

TERMINAL

```
npx skills add https://github.com/Leonxlnx/taste-skill

# Or install a single named skill from the collection:
npx skills add https://github.com/Leonxlnx/taste-skill \
--skill "design-taste-frontend"

# Activate explicitly in a session:
/skill taste-skill
Build me a landing page for a design agency.
```

The repository also documents a bash-script installer. Prefer the `npx skills add` path above and read Section 01 before using any installer that pipes directly into a shell.

SECTION 05

Side by side

The honest differences, so you install based on what you actually need rather than the order they were mentioned in a reel.

	Emil Kowalski's skill	Impeccable	Taste Skill
Core focus	Motion and interaction specifically	Whole interface — typography, color, spacing, layout, motion	Overall aesthetic direction and taste, continuously applied
Interaction model	Triggers automatically on animation-related tasks; several named sub-skills for specific jobs	Explicit commands you run at specific moments	A standing point of view, plus an explicit /skill invocation
Best single use	"Review the animations on this page" or "should this even animate?"	"/polish this before I ship it"	"Build this in a brutalist, high-contrast direction"
Depth on motion	The deepest of the three — this is its whole subject	Covered as one of seven pillars	Covered via the MOTION_INTENSITY dial
Depth on visuals	Light — mentioned, not the focus	The deepest of the three — most of its seven pillars	Broad coverage, less prescriptive than Impeccable
Setup step	None required	/impeccable init writes project context once	None required, though stating a direction up front helps

If you can only install one

Impeccable, for most people — it is the broadest single skill and includes a motion pillar, even if it goes shallower than Emil Kowalski's skill specifically on animation. Add Emil Kowalski's skill next if motion-heavy interfaces are a real part of your work; add Taste Skill next if you want named aesthetic directions rather than one house style.

SECTION 06

Running all three together

They do not conflict by design, but they were built independently and were never tested against each other by their authors. This order avoids the friction points.

The order that works

01 Set direction first, with Taste Skill. Before any code exists, decide the aesthetic — "build this landing page in a premium, editorial direction." This shapes everything downstream.

02 Build with Impeccable's structure. Run `/impeccable init` once per project so typography, color and spacing follow a consistent system as pages get built.

03 Bring in Emil Kowalski's skill for motion specifically — once the static layout is right, ask it to find animation opportunities and apply them, or review what Impeccable's `/animate` already added.

04 Close with /polish and /critique from Impeccable as the final pass, since it is the broadest of the three and best positioned to catch inconsistency across everything the other two touched.

Where they can step on each other

— Two skills both trying to set the type scale. If Taste Skill establishes a direction and Impeccable's init runs afterward with different defaults, you get two competing systems. Fix: let Taste Skill set direction, then tell Impeccable's init to follow it rather than starting fresh.

— Motion applied twice, differently. If Impeccable's /animate already added transitions and Emil Kowalski's skill reviews afterward, expect it to flag some of what was just added — that is correct behaviour, not a bug. Let the review win; it is the stricter, more specific standard on motion specifically.

— Every installed skill's description sits in context on every turn, whether triggered or not. Three well-scoped skills is a sensible working set — installing many more "just in case" adds token overhead for no benefit, sometimes called context tax. Audit what you actually use every few weeks.

> A useful mental model: Taste Skill decides what the site should feel like, Impeccable makes sure every pixel agrees with that decision, and Emil Kowalski's skill makes sure nothing moves without a reason.

SECTION 07

Prompts that use all three well

The skills trigger automatically in many cases, but naming them explicitly gets more consistent results — especially early on, before you have a feel for what triggers what.

Starting a new page

PROMPT

```
Build a [PAGE TYPE] for [BUSINESS/PROJECT]. Aesthetic direction:
[e.g. premium, editorial, high-contrast] — use taste-skill for the
overall direction.

Follow the project's DESIGN.md for typography, color and spacing
(Impeccable). Once the static layout is solid, find genuine
animation opportunities and apply them following Emil Kowalski's
animation rules - nothing over 300ms, transform/opacity only,
and skip anything that would animate just for the sake of it.
```

Polishing something that already exists

PROMPT

```
/audit this page first and tell me what's off before changing
anything.

Then /polish it.

Then review the animations against Emil Kowalski's rules and fix
anything over 300ms or animating a layout property instead of
transform/opacity.
```

Finding out what to animate at all

PROMPT

```
Use find-animation-opportunities to scan this page. Show me what
you'd add motion to and what you'd deliberately leave still,
with your reasoning for each, before implementing anything.
```

SECTION 08

How to tell it's working

Concrete, checkable differences — not a feeling.

- [ ] The typeface is not Inter by default, or Inter is a deliberate choice you can justify
- [ ] There is one accent color used with restraint, not a gradient doing the work of a whole palette
- [ ] No card is nested inside another card without a reason
- [ ] Spacing follows a real scale — sections have room to breathe, not uniform 24px gaps everywhere
- [ ] Animations are fast (under ~300ms) and only on transform or opacity
- [ ] At least one interface element was deliberately left unanimated, and you can say why
- [ ] The mobile layout was designed, not the desktop layout shrunk
- [ ] You could describe the site's aesthetic direction in one phrase, and every page matches it

What does NOT mean it failed

A /polish or /audit pass that returns a long list of issues on the first run is normal — that is the skill doing its job on default AI output. Judge it by the second pass, after fixes, not the first report.

APPENDIX

Sources & notes

Checked July–August 2026, directly from each project's own repository or site. All three are actively maintained and change fast — re-check the source before relying on exact command names months from now.

What the guide says	Source
Emil Kowalski is a design engineer at Linear, creator of Sonner and Vaul, author of Animations on the Web	emilkowal.ski/skill and the skill's own repository
Install command and the six named skills in the family	github.com/emilkowalski/skill and emilkowal.ski/skill
Core animation rules (300ms guideline, transform/opacity only, interruptibility, reduced-motion)	github.com/emilkowalski/skills — review-animations SKILL.md
Impeccable built by Paul Bakaus, extends Anthropic's official frontend-design skill	Multiple independent 2026 reviews, cross-checked; apidog.com and chaseai.io write-ups
Impeccable's command list, seven design pillars, and /impeccable init flow	mdskills.ai and claudepluginhub.com plugin listings
Taste Skill built by Leon Lin (Leonxlnx), MIT licensed, install commands	github.com/Leonxlnx/taste-skill and tasteskill.dev
Taste Skill's named directions and MOTION_INTENSITY / VISUAL_DENSITY dials	claudemod.com and skillsllm.com skill listings
Skills as software: tool misuse and data exposure risk, treat installation like installing software	Anthropic Agent Skills documentation, platform.claude.com

None of the three skills profiled here are official Anthropic products; they are independent, open-source projects built on top of Claude Code's skill system. Popularity figures (stars, installs) change quickly and are not repeated here for that reason — check each repository directly for current numbers.

THAT'S THE THREE

Taste isn't magic. It's just written down, for once.

Three different people looked at the same problem — AI-generated interfaces defaulting to the same generic look — and each wrote down a different, specific point of view to fix it. None of them make the model smarter. They make it less average, on purpose.

Start with Impeccable if you install only one. Add the other two once you know specifically where your builds are still falling short.

Sanchit Pandey

AI systems, automation and web development.

If you find a fourth skill worth this list, send it over — properly reviewed additions are the only kind this document will ever get.